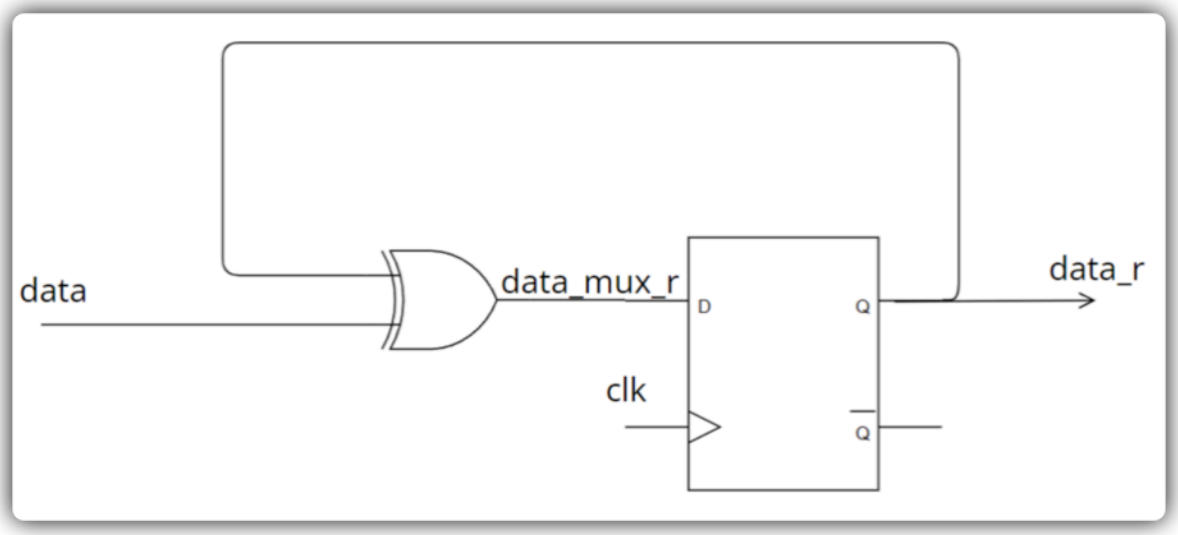
module 12: 单bit跨时钟域

1. 单bit跨时钟域

- 使用Verilog语言，将单bit宽度为10ns的data信号由频率为周期为10ns的时钟域同步至周期为7ns的时钟域(慢到快)
- 使用Verilog语言，将单bit宽度为7ns的data由频率为周期为7ns的时钟域同步到周期为20ns的时钟域，确保同步后的频率展宽为一个时钟周期(快到慢)

2. 单bit跨时钟域原理

- 对于将信号从“快时钟域同步到慢时钟域”对于题目2来说，7ns的data宽度有可能正好处在周期为20ns的信号的两个上升沿之间，这就造成了信号有可能采空的情况出现。
- 将一个脉冲信号展宽为电平信号，让一个很短的脉冲信号变成一个很长的电平信号，那么不就一定能被快时钟域采到了。
- 在慢时钟域重新通过组合逻辑设计寄存器外围电路,这里我们使用异或门来进行展宽。
- 通过边沿检测将电平信号转换回脉冲信号



- 电平同步，脉冲同步，边沿同步
  - 电平同步就是正常的打两拍
  - 边沿同步是在正常的打两拍的基础上，clkb时钟域用边沿检测电路处理一下output信号，让这个信号保证为一个周期

- 脉冲同步是指在clka时钟域将脉冲信号处理成电平信号，再进行打两拍，再重复边沿同步的边沿检测电路的操作。

3. RTL:慢到快

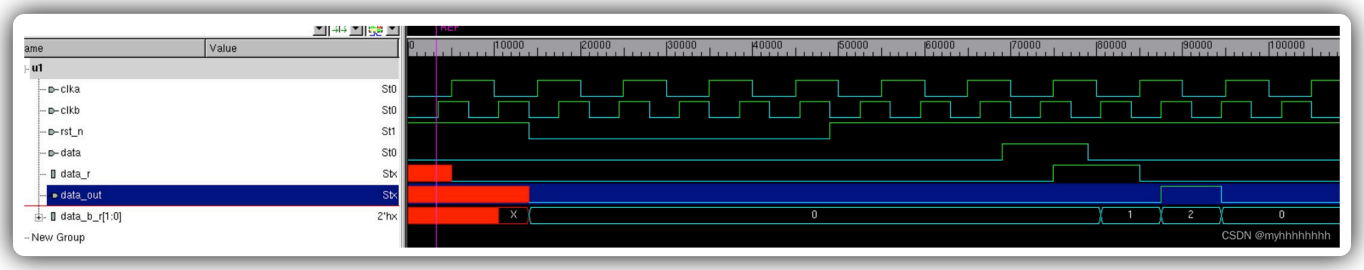
```
module slow2fast_cdc (
    input   i_clk_s,
    input   i_pluse_s,
    input   i_clk_f,
    output  o_pluse_f
);

reg r_pluse_s_d0;
reg r_pluse_s_d1;

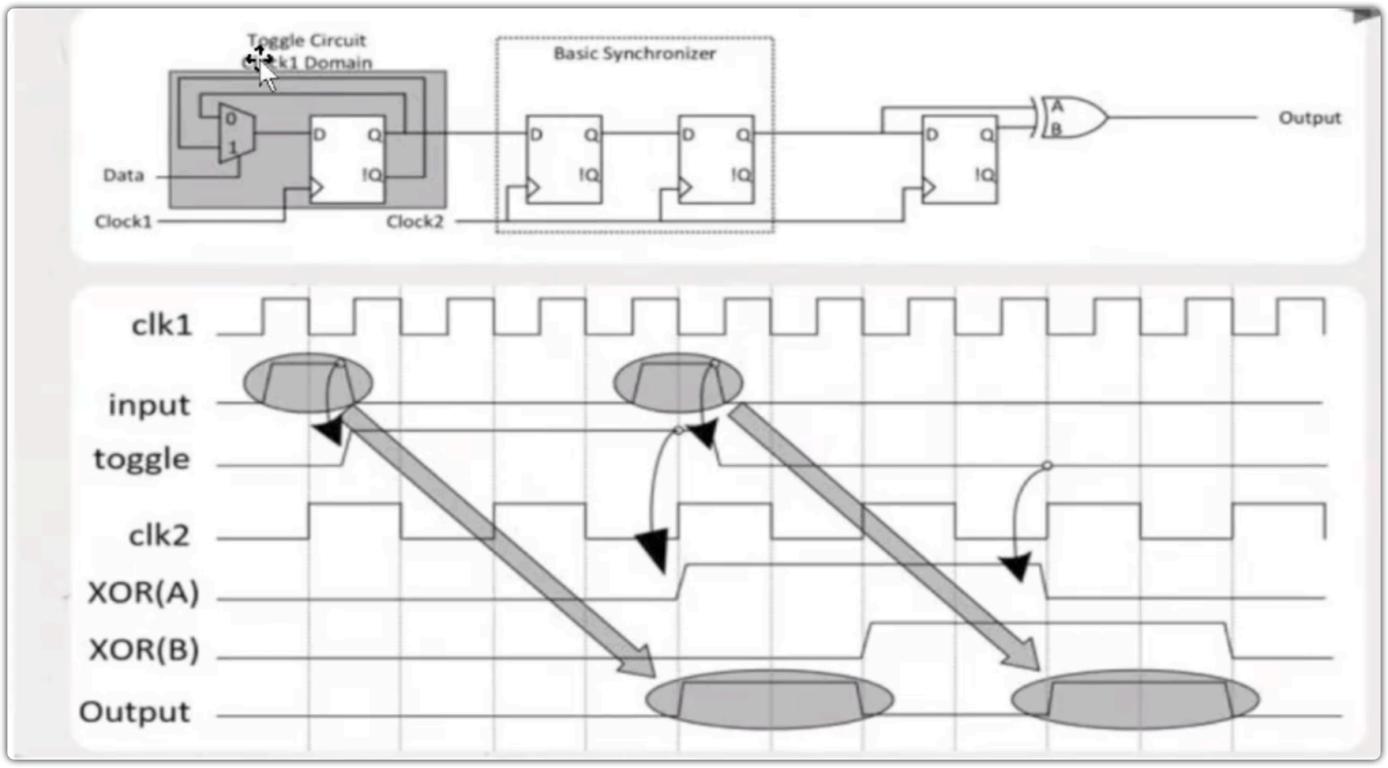
always @(posedge i_clk_f) begin
    r_pluse_s_d0 <= i_pluse_s;
    r_pluse_s_d1 <= r_pluse_s_d0;
end

assign o_pluse_f = r_pluse_s_d1;

endmodule
```



4. RTL:快到慢




```
module fast2slow_cdc (
   input  i_clk_f,
   input  i_pluse_f,
   input  i_clk_s,
   output o_pluse_s
);

   reg [2:0] r_pluse;
   always @(posedge i_clk_f) begin
      r_pluse <= {r_pluse[1:0], i_pluse_f};
   end

   wire w_pluse;
   assign w_pluse = |r_pluse; //r_pluse[2] | r_pluse[1] | r_pluse[0]

   reg r_pluse_de;
   reg r_pluse_d1;
   always @(posedge i_clk_s) begin
      r_pluse_de <= w_pluse;
      r_pluse_d1 <= r_pluse_de;
   end

   assign o_pluse_s = r_pluse_d1;

endmodule

module fast2slow_cdc2 (
   input   i_clk_f,
   input   i_pluse_f,
   input   i_clk_s,
   output  o_pluse_s
);

reg r_temp = 0;

always @(posedge i_clk_f) begin
   if (i_pluse_f)
      r_temp <= ~r_temp;
   else
      r_temp <= r_temp;
end

reg r_temp_d0;
reg r_temp_d1;
```

```
reg r_temp_d2;

always @(posedge i_clk_s) begin
    r_temp_d0 <= r_temp;
    r_temp_d1 <= r_temp_d0;
    r_temp_d2 <= r_temp_d1;
end

assign o_pluse_s = r_temp_d2 ^ r_temp_d1;

endmodule
```

